Linux Kernel Memory Management on ARMv8

Clean Interview Notes

Focused on the memory-allocation stack shown in the diagram: kmalloc, vmalloc, slab/SLUB, buddy allocator, zones, memblock, DMA, mmap, and ARMv8 SMP boot flow.

What to remember in one minute	Interview-safe answer
Early boot allocator	memblock is used before the normal page allocator exists.
Runtime page allocator	buddy allocator manages physical pages in powers of two.
Small object allocator	SLUB/SLAB sits on top of buddy and serves kmalloc/kmem_cache allocations.
Large virtually contiguous memory	vmalloc maps scattered physical pages into one contiguous kernel virtual range.
DMA-safe coherent memory	dma_alloc_coherent provides CPU/device shared buffers with coherent semantics.
Userspace memory	mmap creates VMAs; actual pages are often allocated lazily on page fault.
ARMv8 multicore angle	Boot CPU initializes memory management first, then secondary CPUs are brought online and start using per-CPU fast paths.

Question 01. Mermaid flow — how it works

High-level explanation

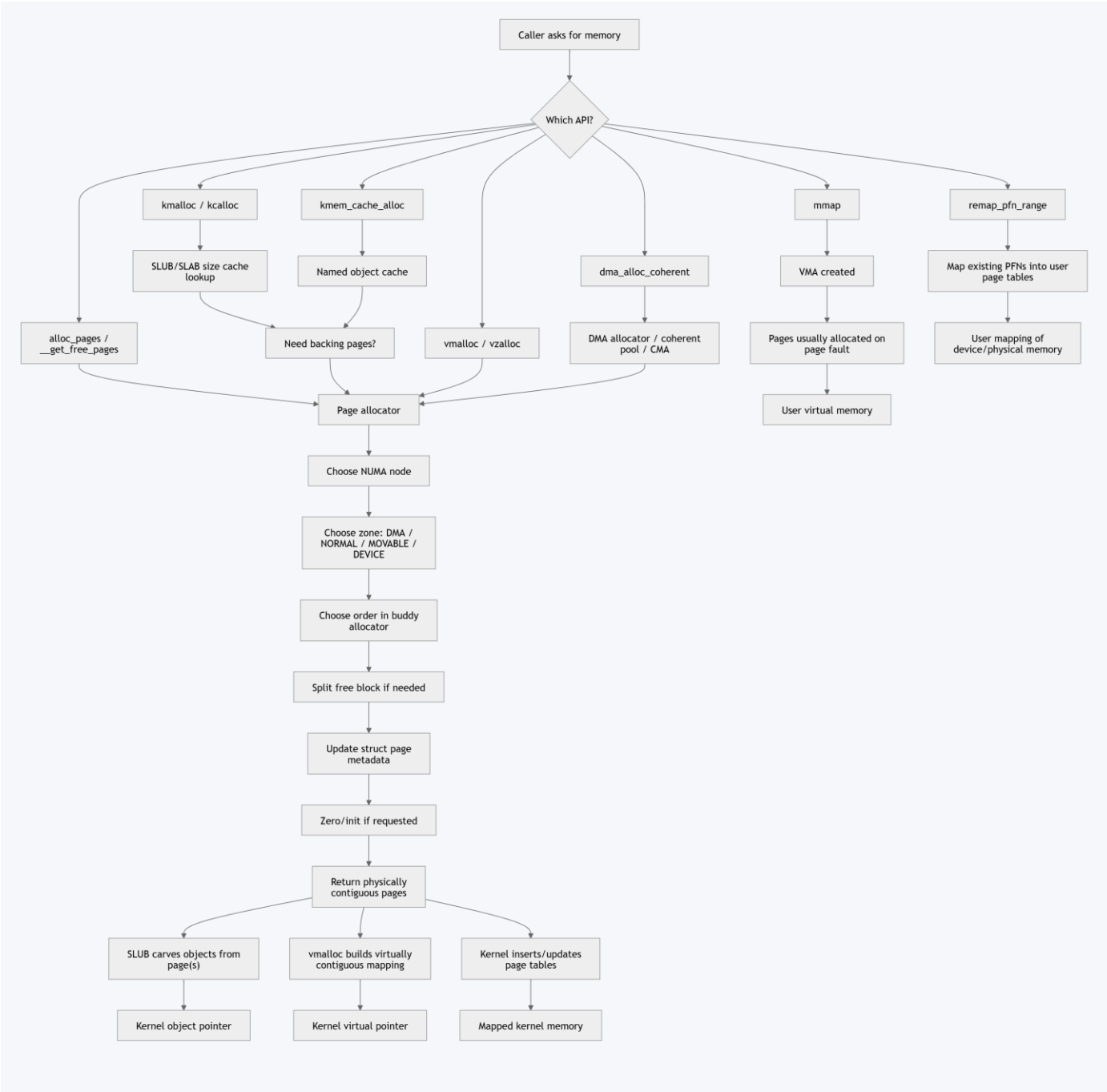
The top layer in the diagram is the API layer exposed to kernel code and to userspace-facing mechanisms. Those APIs do not all allocate memory in the same way. Instead, each one chooses the most appropriate lower layer. Small kernel objects normally go through SLUB/SLAB, raw pages go through the buddy allocator, large virtually contiguous regions go through vmalloc, and DMA-safe shared buffers go through the DMA API. At the bottom, all normal RAM management eventually depends on page metadata, zones, free lists, reclaim, and compaction.

Mental model

- Request comes in: kernel code asks for memory by intent, not by implementation.
- Allocator selects a path: object allocation, page allocation, virtual mapping, DMA mapping, or user mapping.

- Kernel resolves the request into real pages: choose node, choose zone, choose order, split or reuse pages, update metadata, zero if required.
- Address is returned in the right form: kernel virtual pointer, vmalloc pointer, or userspace mapping.

Mermaid flowchart



Interview points to emphasize

- kmalloc gives physically contiguous memory in the kernel direct map for sizes that fit its allocation path well.
- vmalloc gives virtually contiguous memory, not necessarily physically contiguous memory.
- alloc_pages is the direct page-level interface when the caller wants page granularity.
- remap_pfn_range is mapping existing physical frames, not normal anonymous allocation.

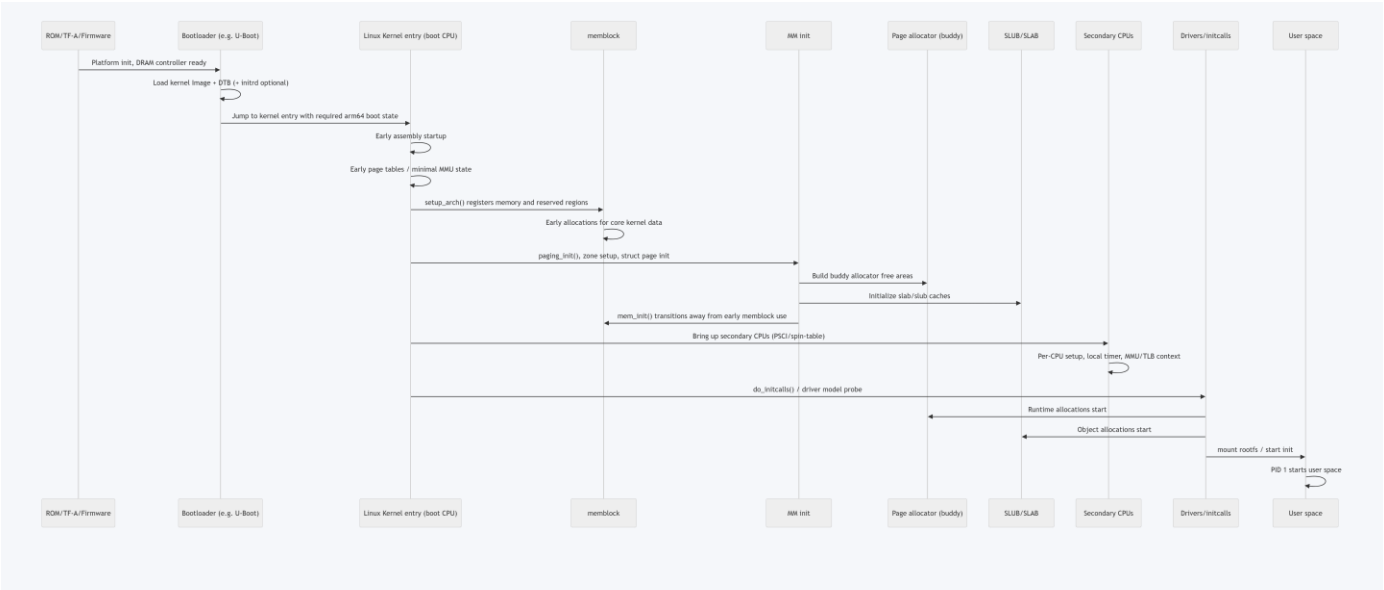
- The buddy allocator owns blocks of pages; SLUB/SLAB owns objects carved from those pages.

Question 02. Boot-time sequence — who initializes what in Linux

Boot story in structured order

- Firmware and bootloader prepare DRAM, load the kernel image, and pass the device tree or equivalent boot information.
- The boot CPU enters the Linux kernel first. Early assembly code establishes minimal execution state and early page tables.
- Architecture setup registers memory regions and reserved regions into memblock. memblock is the early allocator used before the full memory manager is ready.
- The kernel initializes zones, struct page metadata, and the buddy allocator.
- SLUB/SLAB is initialized after the page allocator is usable.
- Only after core memory management is ready are secondary CPUs brought online and drivers fully probed.

Sequence diagram



Who initializes what

Stage	Owner	What gets initialized
Platform bootstrap	Firmware / bootloader	DRAM controller readiness, kernel image placement, boot parameters, entry contract.
Very early kernel	Boot CPU arch code	Exception level assumptions, early stack, early page tables, first MMU view.
Boot-time memory	memblock	Registration of RAM and reserved regions plus early allocations.

Stage	Owner	What gets initialized
Runtime memory core	Core MM	Zones, struct page array, buddy allocator, reclaim foundations.
Object allocator	SLUB/SLAB	kmalloc size caches and specialized object caches.
SMP stage	PSCI / SMP bring-up	Secondary CPUs, per-CPU areas, scheduler-ready state.
Driver/runtime stage	Initcalls and subsystems	DMA allocations, vmalloc, mmap handlers, userspace start-up.

Question 03. Cache-line discussion — where it exists and when it matters

Important clarification

In interview discussions, “cache line” is often used loosely. In this topic it can refer to two different things: (1) allocator-side alignment and object placement inside SLUB/SLAB caches, and (2) CPU data-cache coherency and cache maintenance rules relevant to DMA and ARMv8 memory attributes. Treat them separately.

A. Allocator-side cache-line alignment

- Implemented in practice when slab caches are defined and when kmalloc size classes are created.
- The object layout determines alignment, padding, and how many objects fit in a slab.
- Per-CPU freelists reduce cache-line bouncing between cores by making most small allocations local to the current CPU.
- This is called every time kmalloc or kmem_cache_alloc consumes an object from a cache.

B. CPU cache coherency for DMA on ARMv8

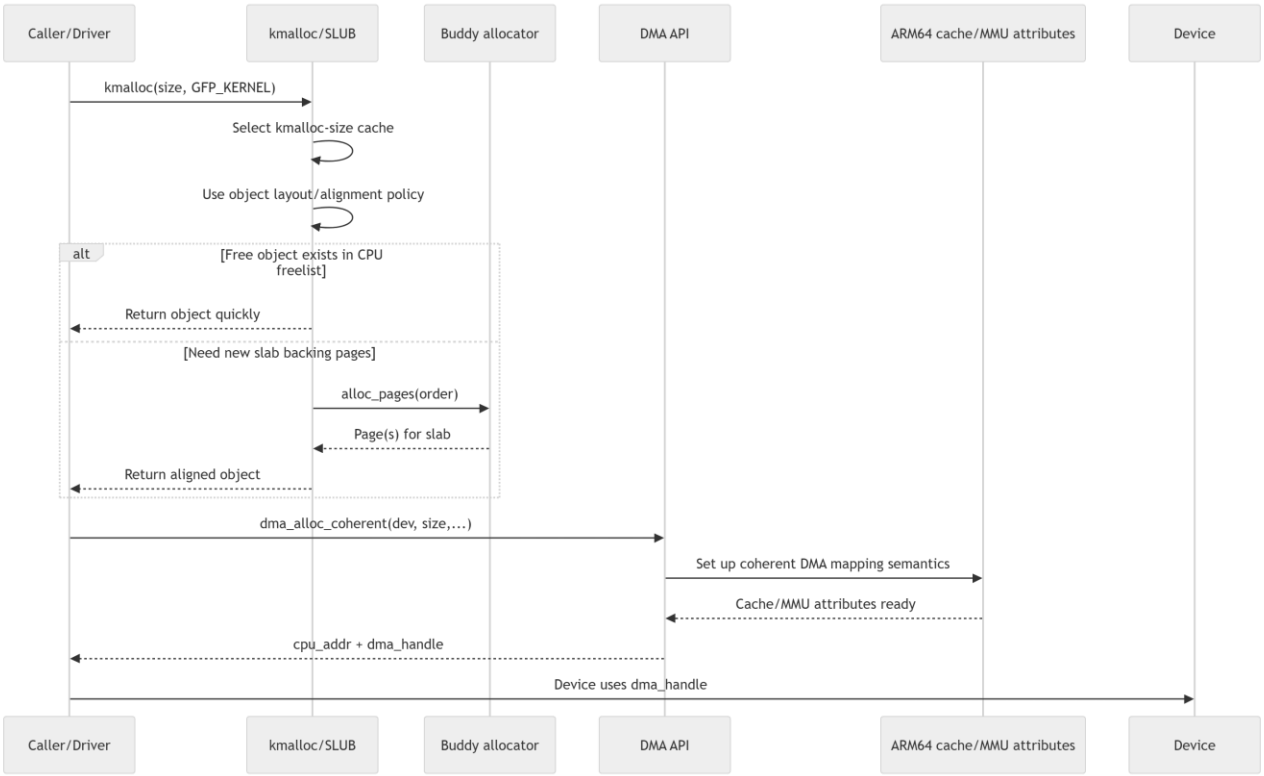
- This is handled through the DMA API and architecture-specific mapping rules, not by SLUB itself.
- dma_alloc_coherent returns memory intended for CPU and device sharing with coherent semantics.
- Streaming DMA paths use dma_map_single, dma_map_page, dma_unmap, and explicit sync operations where needed.
- On ARMv8, page attributes and the device coherency model determine whether cache maintenance is needed and when barriers matter.

Representative call-path sketch

```
Case 1: kmalloc(128, GFP_KERNEL)
    caller
      -> kmalloc
      -> select kmalloc size cache
      -> SLUB fast path: try per-CPU freelist
      -> if empty: fetch partial slab or allocate new slab pages from buddy
      -> return aligned object pointer

Case 2: dma_alloc_coherent(dev, size, &dma_handle, gfp)
    driver
      -> DMA API front-end
```

```
    -> arch / platform DMA backend
    -> coherent pool or CMA or page allocator backing
    -> CPU mapping + device handle returned
```



Interview-safe conclusion

If the interviewer asks “where is cache line implemented?”, the safest answer is: cache-line alignment for small kernel objects is baked into slab-cache layout and per-CPU fast paths, while cache coherency for devices is handled by the DMA API plus architecture-specific mapping rules.

Question 04. Walk-through of major kernel functions and why they matter

Quick function map

Function / API	Purpose	Why it matters in interviews
kmalloc	Fast general-purpose kernel allocation	Default answer for small kernel buffers and structs.
kcalloc	Zero-initialized array allocation	Shows safe allocation plus initialization semantics.
kmem_cache_create / alloc	Dedicated cache for one object type	Explains object reuse, locality, and constructors.

Function / API	Purpose	Why it matters in interviews
alloc_pages(order)	Allocate 2^order contiguous pages	Core page-allocator interface.
__get_free_page	Single-page convenience helper	Historical but easy way to explain order-0 pages.
vmalloc / vzalloc	Virtually contiguous kernel memory	Important contrast with kmalloc.
dma_alloc_coherent	Coherent CPU/device shared memory	Critical for descriptors and rings.
mmap	Create userspace virtual mappings	Explains VMA + lazy fault path.
remap_pfn_range	Map existing PFNs to user space	Used by drivers and reserved-memory mappings.
setup_arch	Architecture-specific boot setup	Where memory gets registered early.
paging_init	Build runtime paging structures	Transitions to the full MMU view.
mem_init	Finish boot-time memory transition	Marks handoff from early boot allocator to runtime state.

Detailed notes by function

kmalloc

Used for kernel heap objects that need to be physically contiguous and efficiently accessible. Usually backed by kmalloc size caches in SLUB. Best answer when someone asks for the normal kernel allocation path.

kcalloc

Like kmalloc but zeroes memory. Good for arrays and safer initialization patterns.

kmem_cache_create / kmem_cache_alloc

Creates a dedicated cache for one object type, such as inode-like or task-like objects. Important when discussing locality, constructors, debug options, and object reuse.

alloc_pages(order)

Requests page blocks directly from the buddy allocator. The order argument means the block size is 2^order pages. This is the right place to explain contiguous physical memory.

__get_free_page / get_zeroed_page

Thin convenience wrappers around order-0 page allocation. Useful interview shorthand for single-page allocation.

vmalloc / vzalloc

Allocates a contiguous virtual range by mapping separately allocated physical pages. Slower than kmalloc because it needs page-table work and TLB effects become relevant.

dma_alloc_coherent

Returns memory with DMA-safe coherent semantics, typically for rings, descriptors, and control structures shared with hardware.

mmap

Creates a userspace VMA. Real pages may be attached immediately or lazily when a fault occurs, depending on mapping type.

remap_pfn_range

Driver-facing helper to expose an existing physical frame range into a userspace VMA. This is mapping, not normal allocation.

setup_arch

Early architecture setup, usually where memory description from firmware or device tree gets registered and reserved areas are carved out.

paging_init

Builds the runtime kernel mapping model and prepares the system to use the normal memory-management view.

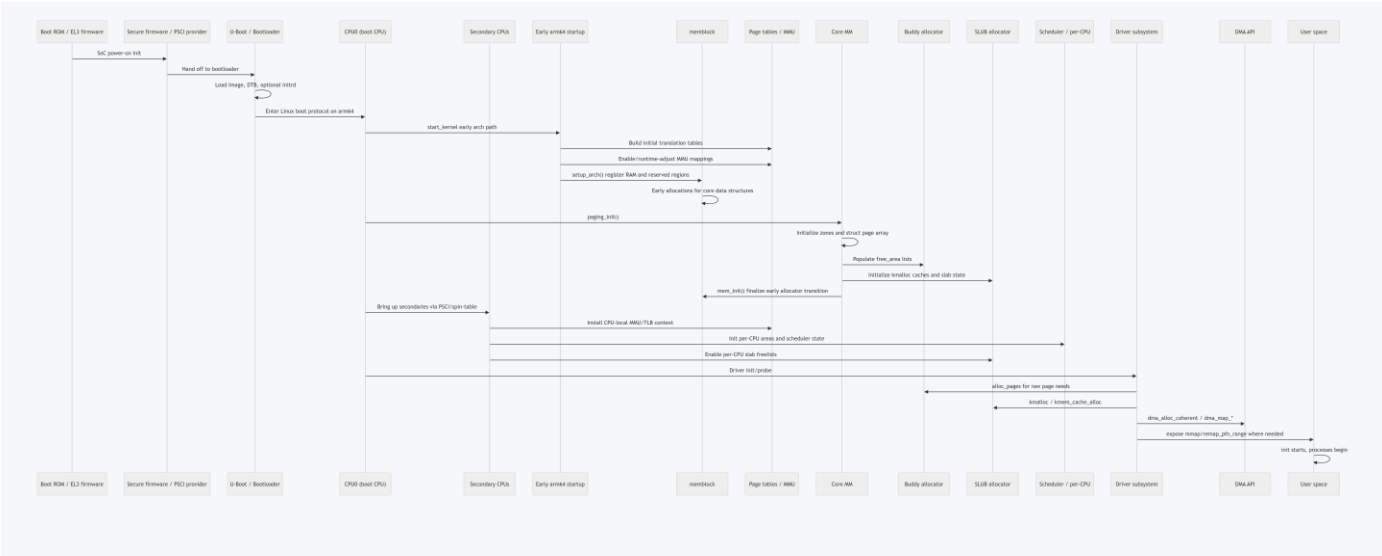
mem_init

Finishes the early-boot memory stage and finalizes the switch away from the boot-time allocator.

Question 05. Complete ARMv8 multi-core + Linux kernel sequence diagram

Full story with multicore context

On ARMv8, the boot CPU enters first and performs the early memory-management work. Secondary cores do not start allocating general kernel memory until the core MM state is ready. Once secondary CPUs are online, per-CPU allocator fast paths, scheduler state, cache-line locality, TLB shootdowns, and DMA coherency become much more visible in performance discussions.



ARMv8-specific interview notes

- The early allocator is memblock, not buddy. This is a very common interview checkpoint.
- SMP bring-up is later than the earliest MM initialization; the boot CPU prepares the ground first.
- Per-CPU slab freelists matter on multicore systems because they reduce lock traffic and cache-line contention.
- DMA coherency must be explained separately from slab caching. Many candidates accidentally mix these topics.
- High-order contiguous allocation becomes harder as fragmentation grows, which is why compaction and CMA matter in long-running systems.

Recommended answer structure in a real interview

- Start from the top: “The API chosen depends on the memory intent.”
- Then separate boot-time memory from runtime memory.
- Then explain buddy versus SLUB in one clean sentence.
- Then contrast kmalloc with vmalloc and alloc_pages.
- Then close with ARMv8 multicore specifics: boot CPU first, secondary CPUs later, per-CPU allocator paths, DMA coherency, and TLB/cache considerations.

Common mistakes to avoid

- Do not say vmalloc is physically contiguous.
- Do not say mmap always allocates pages immediately.
- Do not confuse slab “cache” with CPU data cache coherency.
- Do not skip memblock when explaining boot.
- Do not claim all DMA buffers are created through kmalloc; use the DMA API explanation instead.

Glossary

Term	Meaning
memblock	Early-boot memory allocator and region tracker used before the normal page allocator is active.
buddy allocator	Runtime physical page allocator organized by block order, where each order is a power-of-two number of pages.
SLUB/SLAB	Kernel object allocator built on top of page allocations.
struct page	Per-page metadata describing state, flags, mapping, and reference information.
zone	Memory partition with allocator constraints such as DMA reachability or movement policy.
VMA	Virtual memory area representing a userspace

Term	Meaning
	mapping region.
CMA	Contiguous Memory Allocator used to help satisfy DMA or large contiguous allocation needs.
TLB	Translation lookaside buffer caching page-table translations in the CPU.